

AFRILANG Stdlib

std/c/cryptcaesar

Français. Bibliothèque AFRILANG 'std/c/cryptcaesar' (niveau complexe, catégorie complexe). Elle expose 6 fonction(s) : encrypt, decrypt, bruteForce, countAlpha, rotAll, isCaesar.

```
'import "std/c/cryptcaesar"' · 'use cryptcaesar'
```

- 'encrypt(s text, shift number) → text'
- 'decrypt(s text, shift number) → text'
- 'bruteForce(s text) → list text'
- 'countAlpha(s text) → number'
- 'rotAll(s text) → list text'
- 'isCaesar(plain text, cipher text) → bool'

English. AFRILANG library 'std/c/cryptcaesar' (tier complex, category complex). Exports 6 function(s): encrypt, decrypt, bruteForce, countAlpha, rotAll, isCaesar.

```
'import "std/c/cryptcaesar"' · 'use cryptcaesar'
```

- 'encrypt(s text, shift number) → text'
- 'decrypt(s text, shift number) → text'
- 'bruteForce(s text) → list text'
- 'countAlpha(s text) → number'
- 'rotAll(s text) → list text'
- 'isCaesar(plain text, cipher text) → bool'

Catégorie / Category	Modules complexe (c/)	
Niveau / Tier	complexe	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/cryptcaesar' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : encrypt, decrypt, bruteForce, countAlpha, rotAll, isCaesar.

```
'import "std/c/cryptcaesar"' · 'use cryptcaesar'
```

```
- `encrypt(s text, shift number) → text`
- `decrypt(s text, shift number) → text`
- `bruteForce(s text) → list text`
- `countAlpha(s text) → number`
- `rotAll(s text) → list text`
- `isCaesar(plain text, cipher text) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptcaesar"
use cryptcaesar
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- encrypt(s text, shift number) → text•
- decrypt(s text, shift number) → text•
- bruteForce(s text) → list•
- countAlpha(s text) → number•
- rotAll(s text) → list•
- isCaesar(plain text, cipher text) → bool

4. Exemples d'utilisation

```
import "std/c/cryptcaesar"
use cryptcaesar
```

```
create result = encrypt(/* args */)
say result
```

```
create result = decrypt(/* args */)
say result
```

```
create result = bruteForce(/* args */)
say result
```

```
create result = countAlpha(/* args */)
say result
```

```
create result = rotAll(/* args */)
say result
```

```
create result = isCaesar(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/encryptcaesar"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module cryptcaesar

```
function encrypt(s text, shift number) returns text  
end
```

```
function decrypt(s text, shift number) returns text  
end
```

```
function bruteForce(s text) returns list text  
end
```

```
function countAlpha(s text) returns number  
end
```

```
function rotAll(s text) returns list text  
end
```

```
function isCaesar(plain text, cipher text) returns bool  
end  
end
```

English

1. Overview

AFRILANG library 'std/c/cryptcaesar' (tier complex, category complex). Exports 6 function(s): encrypt, decrypt, bruteForce, countAlpha, rotAll, isCaesar.

```
'import "std/c/cryptcaesar"' · 'use cryptcaesar'
```

```
- `encrypt(s text, shift number) → text`  
- `decrypt(s text, shift number) → text`  
- `bruteForce(s text) → list text`  
- `countAlpha(s text) → number`  
- `rotAll(s text) → list text`  
- `isCaesar(plain text, cipher text) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptcaesar"  
use cryptcaesar
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- encrypt(s text, shift number) → text•
decrypt(s text, shift number) → text•
bruteForce(s text) → list•
countAlpha(s text) → number•
rotAll(s text) → list•
isCaesar(plain text, cipher text) → bool

4. Usage examples

```
import "std/c/cryptcaesar"  
use cryptcaesar
```

```
create result = encrypt(/* args */)   
say result
```

```
create result = decrypt(/* args */)   
say result
```

```
create result = bruteForce(/* args */)   
say result
```

```
create result = countAlpha(/* args */)   
say result
```

```
create result = rotAll(/* args */)   
say result
```

```
create result = isCaesar(/* args */)   
say result
```

5. Best practices

- Prefer ``import "std/c/encryptcaesar"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module cryptcaesar

```
function encrypt(s text, shift number) returns text
end
```

```
function decrypt(s text, shift number) returns text
end
```

```
function bruteForce(s text) returns list text
end
```

```
function countAlpha(s text) returns number
end
```

```
function rotAll(s text) returns list text
end
```

```
function isCaesar(plain text, cipher text) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/encryptcaesar"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/encryptcaesar/>

Source (extrait)

```
module cryptcaesar

function encrypt(s text, shift number) returns text
end

function decrypt(s text, shift number) returns text
end

function bruteForce(s text) returns list text
end

function countAlpha(s text) returns number
end

function rotAll(s text) returns list text
```

```
end
```

```
function isCaesar(plain text, cipher text) returns bool
```

```
end
```

```
end
```